

Build the dataset

Coding Lesson 2 · Math Camp 2026

Rony Rodriguez-Ramirez

Harvard Kennedy School · API 209

Wednesday, August 19, 2026

Lesson map

From a question to a table

Define the object

Six data verbs

Joining tables

Build and record the table

From Lesson 2 to Lab 2

Before we begin

If the analysis needs one row for each country and year, what could produce two rows for the same country and year?

Write for two minutes

Give one possible cause and one check that could expose it.

We will return to these answers when we discuss joins.

The goal for Lesson 2

Turn one policy question into a checked analysis table.

The table should have a clear population, unit, key, period, variables, and record of the checks we performed.

What I want you to do by the end

1. Translate a table contract into readable R.
2. Use six data verbs for a stated reason.
3. Predict whether rows, columns, values, or the unit should change.
4. Test a country-year key before and after a join.
5. Record how the table was built and checked.
6. Ask an agent for one improvement that you can evaluate.

Route for the lesson

Minutes	Work	Main question
00-25	Contract + inspect	What table do we need?
25-60	Select + filter	Which variables and observations remain?
60-85	Mutate + group	What value or unit did we create?
85-103	Join	Did any row multiply or fail to match?
103-113	Agent pass	Which bounded check can it improve?
113-120	Build record	Can another person inspect our table?

The question comes before the verbs

Before we touch R, write:

- ▶ the population;
- ▶ the unit of observation;
- ▶ the key;
- ▶ the time period;
- ▶ the outcome;
- ▶ the comparison and grouping variables.

We call this the **table contract**. It describes the object that the code should produce.

The health table contract

Field	Decision	Why it matters
Population	Countries and economies with observed indicators	Defines who can appear.
Unit	Country-year	Defines what one row means.
Key	iso3c + year	Must identify every row once.
Period	2000–2022	Defines the comparison window.
Outcome	Under-five mortality	Names the quantity to describe.
Comparison	GDP per capita, PPP	Names the relationship of interest.

This is a descriptive exercise with World Development Indicators ([World Bank, 2026](#)).

Read the frozen file

```
library(tidyverse)

wdi <- read_csv(
  "data/derived/math-camp-wdi-2000-2022.csv",
  show_col_types = FALSE
)

glimpse(wdi)
```

Keep the frozen file unchanged and build forward with code that can be run again (Wickham et al., 2023).

Look before you transform

Four small questions:

1. What does one row represent?
2. Which columns identify that row?
3. Are the identifiers unique?
4. Which years and countries are present?

`glimpse()`, `names()`, `count()`, and a short `summarise()` can answer these structural questions compactly.

Record the source structure

```
wdi |>
  summarise(
    rows = n(),
    countries = n_distinct(iso3c),
    first_year = min(year),
    last_year = max(year),
    duplicate_keys = anyDuplicated(data.frame(iso3c, year))
  )
```

A structure check tells us whether the source has the rows, variables, and keys that we expected.

Lesson map

From a question to a table

Six data verbs

Predict, run, check, explain

Joining tables

Build and record the table

From Lesson 2 to Lab 2

Six verbs, six decisions

Verb	Question before we run it
<code>select()</code>	Which variables and identifiers remain?
<code>filter()</code>	Which observations leave the population?
<code>mutate()</code>	What new value is created, and in which unit?
<code>group_by()</code>	Which observations belong together?
<code>summarise()</code>	What does one output row represent?
<code>left_join()</code>	Which keys connect the tables?

Use one recurring move



Predict rows, columns, values, order, or unit before running the step.

select() defines the scope

```
health <- wdi |>
  select(
    iso3c,
    country,
    year,
    income_level_current,
    gdp_per_capita_ppp,
    under5_mortality
  )
```

Predict: Which columns will leave? Which identifiers need to stay?

Check after select()

```
names(health)
glimpse(health)

stopifnot(all(
  c("iso3c", "year") %in% names(health)
))
```

Run the key check after confirming that the contract identifiers are still present.

filter() changes the population

```
health <- health |>  
  filter(year >= 2000)
```

Before running:

- ▶ How many years should remain?
- ▶ Could a country disappear completely?
- ▶ Are we removing missing indicators on purpose or by accident?

Check after filter()

```
health |>
  summarise(
    rows = n(),
    countries = n_distinct(iso3c),
    first_year = min(year),
    last_year = max(year)
  )
```

Compare country and year coverage to identify which observations left the sample.

mutate() makes a claim about meaning

```
health <- health |>
  mutate(
    log_income = log(gdp_per_capita_ppp)
  )
```

The logarithm compresses the upper tail. It also changes how we interpret a later coefficient. The new column carries a substantive decision about scale.

Check after mutate()

```
health |>
  summarise(
    missing_income = sum(is.na(gdp_per_capita_ppp)),
    missing_log = sum(is.na(log_income)),
    min_income = min(gdp_per_capita_ppp, na.rm = TRUE),
    max_income = max(gdp_per_capita_ppp, na.rm = TRUE)
  )
```

What would happen if income were zero or negative?

Grouping changes the unit

```
regional_year <- health |>
  group_by(region, year) |>
  summarise(
    mean_mortality = mean(under5_mortality, na.rm = TRUE),
    countries = n_distinct(iso3c),
    .groups = "drop"
  )
```

Grouping produces one region-year row in this summary.

Predict the output

Suppose the table has seven regions and 23 years.

1. What is the largest possible row count after the summary?
2. Why might the actual count be smaller?
3. Does `na.rm = TRUE` solve missing coverage?

Take three minutes. Write the answers before we run the code.

Lesson map

From a question to a table

Six data verbs

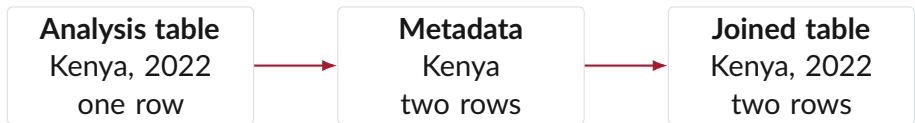
Joining tables

Protect the unit

Build and record the table

From Lesson 2 to Lab 2

A join can multiply rows quietly



Every value can look real while Kenya receives twice the weight. Inspect the join structure because the values alone will hide this problem.

Test the right-side key first

```
stopifnot(  
  !anyDuplicated(metadata["iso3c"])  
)  
  
joined <- left_join(  
  health,  
  metadata,  
  by = "iso3c"  
)
```

The key on the right table determines whether rows on the left can multiply.

Leave a join record

```
stopifnot(nrow(joined) == nrow(health))
stopifnot(!anyDuplicated(joined[c("iso3c", "year")]))

unmatched <- anti_join(
  health,
  metadata,
  by = "iso3c"
)
```

Successful syntax still requires row-count and key checks to confirm that the join preserved the object described by the table contract.

Diagnose the broken join

The row count increased after `left_join()`.

Which check should come first?

1. Delete duplicated rows in the output.
2. Test the right-side key for duplicates.
3. Change to `inner_join()`.
4. Ask an agent to rewrite the complete pipeline.

The first useful check is number 2. The other options can hide the reason.

Lesson map

From a question to a table

Six data verbs

Joining tables

Build and record the table

One common path

From Lesson 2 to Lab 2

One complete health pipeline

```
health <- wdi |>
  select(
    iso3c, country, year, income_level_current,
    gdp_per_capita_ppp, under5_mortality
  ) |>
  filter(year >= 2000) |>
  mutate(log_income = log(gdp_per_capita_ppp))

stopifnot(!anyDuplicated(health[c("iso3c", "year")]))
stopifnot(all(health$year >= 2000))
```

Read the pipeline as decisions

1. **Population:** rows before 2000 leave the sample.
2. **Scope:** selected columns define the health question.
3. **Transformation:** logged income stays beside the original measure.
4. **Structure:** the country-year key is tested in code.

Each important line should correspond to a field in the table contract.

Use current metadata within its time meaning

`income_level_current` is the classification attached when this snapshot was assembled.

It is useful for a current grouping. It does **not** prove that a country belonged to that income group in 2000.

A historical classification needs a separate country-year source, a retrieval date, and documentation.

The build record

Rows and key

Input and output rows;
country-year uniqueness.

Missingness

Observed and missing
values by variable.

Matches

Unmatched and multiplied
join keys.

Years

First year, last year, and
coverage.

Ranges

Plausible values in
documented units.

Decision note

What changed and why it
belongs.

The record is useful when every check points back to the table contract.

Lesson map

From a question to a table

Six data verbs

Joining tables

Build and record the table

From Lesson 2 to Lab 2

Bounded help

Give the agent one task you can evaluate

A broad request hides the important decisions:

Too broad

Clean this dataset and make it analysis-ready.

Ask for one improvement:

- ▶ add a key or range assertion;
- ▶ diagnose unmatched join keys;
- ▶ produce a compact build record.

First write a small pipeline that you can read. Then use the agent as a reviewer.

A bounded request for the health table

Goal: Add a build record for the health analysis table.

Context: Read code/02-build-analysis.R and the indicator dictionary.

Permission: Propose a diff. Do not run it or change frozen data.

Constraints: Keep the country-year unit and lowercase names.

Proof: Report rows, unique keys, years, missingness, ranges, unmatched joins, and every failed condition.

Review the proposed diff

For every added check, answer:

1. Which table-contract field does it test?
2. What failure would it expose?
3. Can I explain the output?
4. Does the proposed change affect the population, transformation, or key?

A change to the population, transformation, or key is a substantive decision. The analyst remains responsible even when the request is described as code cleanup.

Decisions the analyst keeps

- ▶ The meaning of one row.
- ▶ The population represented by a filter.
- ▶ Whether a transformation fits the policy question.
- ▶ Whether current metadata can be interpreted historically.
- ▶ Whether missing coverage weakens the comparison.

The analyst interprets the evidence that the agent exposes.

Updating public data is an extension

Extending one indicator beyond 2022 can be useful. It also adds:

- ▶ an internet and API dependency;
- ▶ retrieval dates and version changes;
- ▶ possible revisions to earlier years;
- ▶ new metadata questions.

First build the frozen table correctly. Then extend it with a separate record.

What we learned in Lesson 2

- ▶ The table contract comes before the data verbs.
- ▶ Every verb should have a reason and a check.
- ▶ A join must preserve the intended key and unit.
- ▶ The analysis table should rebuild from a protected source.
- ▶ The agent improves one bounded part under analyst review.

Build one country-year table.

Test its contract and explain the decisions that produced it.

Lab 2 route

Minutes	Work	Result
00–10	Contract	Population, unit, key, period, and variables.
10–25	Select + filter	Required columns and sample.
25–35	Mutate + group	One justified variable.
35–45	Join	Metadata after a key check.
45–52	Agent pass	One reviewable improvement.
52–60	Build record	Checks and a short explanation.

The lab is done when

- ▶ The script rebuilds the table from the frozen file.
- ▶ The country-year key is unique.
- ▶ The build record includes rows, years, missingness, ranges, and matches.
- ▶ You can explain one consequential decision without the agent.

The lab has no submission or grade. Keep the script because the next lesson will use this table.

References I

Wickham, Hadley, Mine Çetinkaya Rundel, and Garrett Grolemund, *R for Data Science*, 2 ed., O'Reilly Media, 2023.

World Bank, “World Development Indicators,” Data catalog 2026.